

Модели непрерывных процессов и гетерогенные модели

Способы описания
и организации

Популяционная модель

типовая модель «хищник-жертва»

На острове бесконтрольно растёт популяция кроликов. Местные фермеры прикладывают значительные усилия, чтобы прекратить увеличение числа кроликов. Для контроля над ситуацией они хотят завезти на остров лисиц.

; модель популяций Лотка-Вольтерра

; сегмент модели непрерывных процессов

Foxes	EQU	80	; количество лис
Rabbits	EQU	1000	; популяция кроликов
K_	EQU	0.2	;эффективность хищника
A_	EQU	0.008	;параметр естественной убыли
B_	EQU	0.0002	;фактор удачной охоты
C_	EQU	0.04	;параметр плодовитости

```
Foxes    INTEGRATE (K_#B_#Foxes#Rabbits - A_#Foxes)
Rabbits  INTEGRATE (C_#Rabbits - B_#Foxes#Rabbits)
```

;сегмент модели дискретных процессов

```
GENERATE    2000
TERMINATE   1
```

Популяционная модель

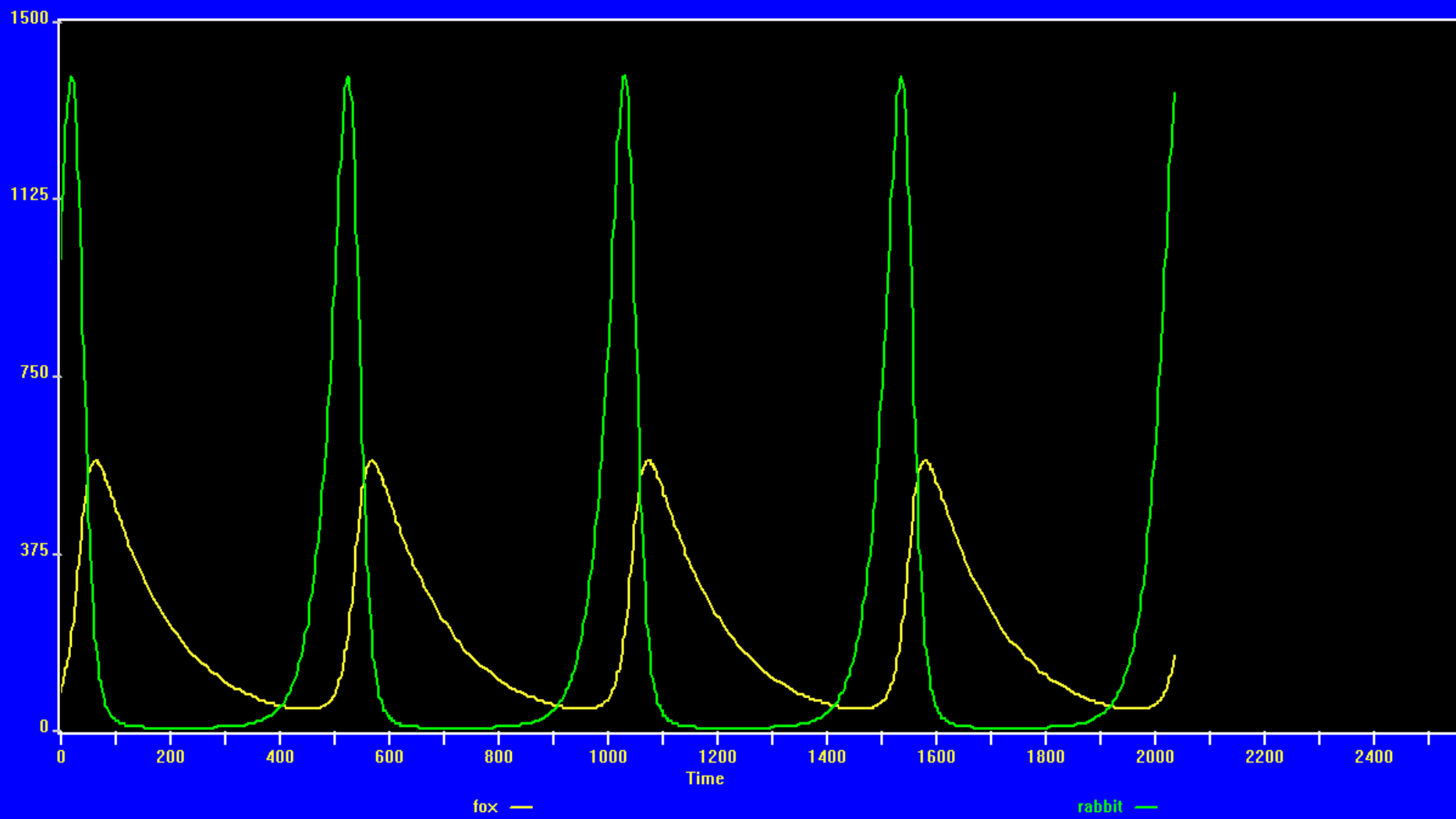


График состояния популяций

Популяционная модель (simpy)

```
class Pool(object):
def __init__(self, env, xinit, yinit):
    self.x = xinit;    self.y = yinit;    self.env = env

    # Процесс ответственный за естественную рождаемость
    def born_x_y(self, alfax, alfay):
        while True:
            self.x += self.x * alfax # прирост жертв
            self.y += self.y * alfay # прирост хищников
            yield self.env.timeout(1)

    # Процесс ответственный за естественную смертность
    def dead_x_y(self, betax, betay):
        while True:
            self.x -= self.x * betax # падёж жертв
            self.y -= self.y * betay # падёж хищников
            if self.x < 2: self.x=2 # остается не менее 2
            yield self.env.timeout(1)
```

Популяционная модель (simpy)

Процесс ответственный за съедение хищником жертв

```
def y_eat_x(self, gammax, gammay):
```

```
    while True:
```

```
        self.x -= self.y * self.x * gammax
```

```
        self.y += self.y * self.x * gammay
```

```
        yield self.env.timeout(1)
```

```
alfax = 0.25    # рождаемость жертв
```

```
alfay = 0.01    # рождаемость хищников
```

```
betax = 0.01    # естественная смертность жертв
```

```
betay = 0.05    # естественная смертность хищников
```

```
gammax = 0.005 # коэф. интенсивности съедания жертв
```

```
gammay = 0.001 # коэф. прироста хищников из-за охоты
```

```
env= simpy.Environment()
```

```
p = Pool(env, 150, 10)    # организация среды
```

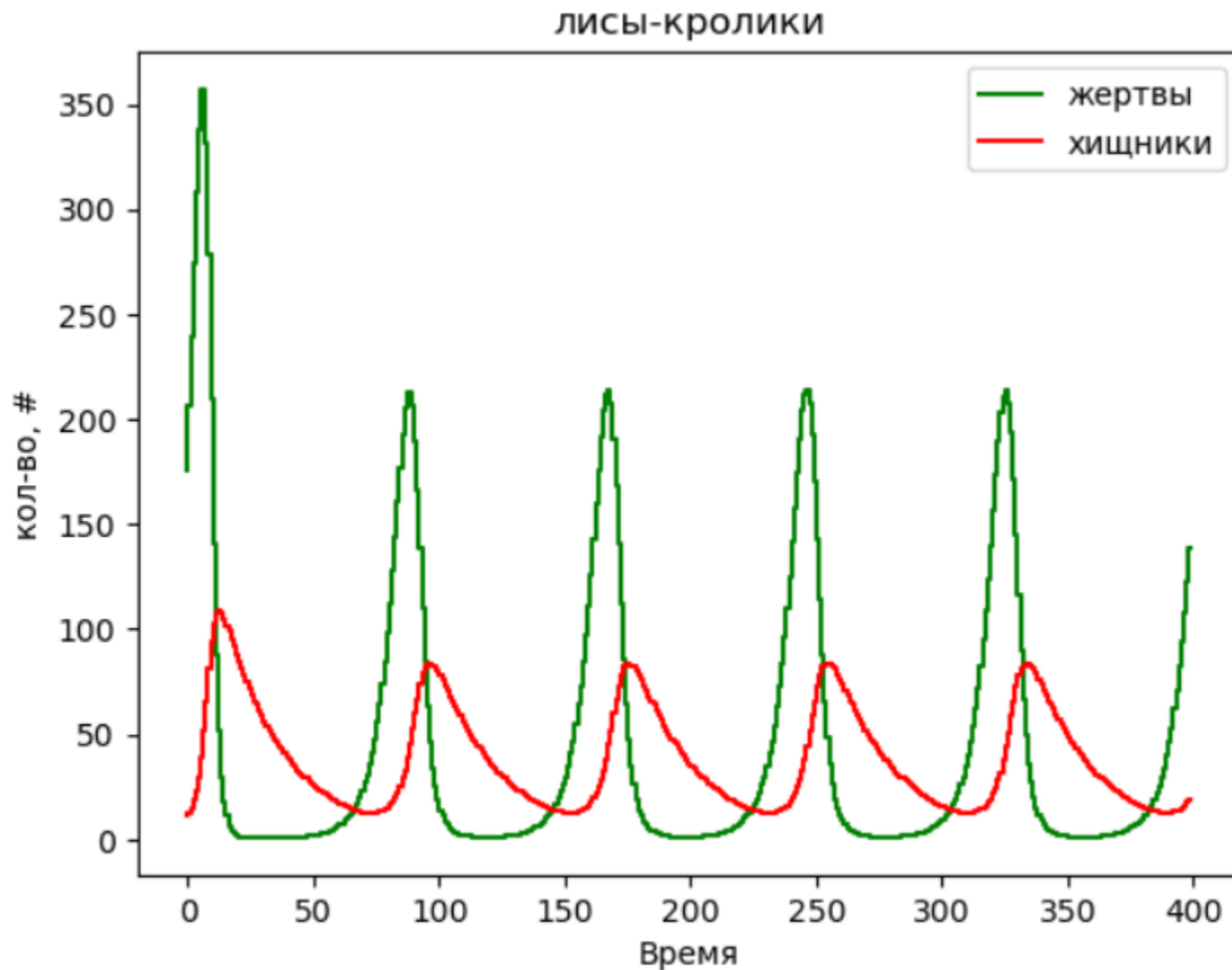
```
env.process(p.born_x_y (alfax,alfay)) # процесс рождения
```

```
env.process(p.dead_x_y (betax,betay)) # процесс гибели
```

```
env.process(p.y_eat_x (gammax, gammay)) # процесс охоты
```

```
env.run(until=400)
```

Популяционная модель (simpy)



Гетерогенная популяционная модель (GPSS)

; модель популяций Лотка-Вольтерра

; сегмент модели непрерывных процессов

Foxes INTEGRATE (K_#B_#Foxes#Rabbits - A_#Foxes),400,maxf

Rabbits INTEGRATE (C_#Rabbits - B_#Foxes#Rabbits),99,minr

;сегмент модели дискретных процессов

minr ADVANCE 15 ;получив сигнал о малом числе кроликов,
plus (DoRabbit(50));выпускаем в лес из зоопарка 50 штук
TERMINATE ;произойдет через 15 дней после сигнала

maxf ADVANCE 25 ;получив сигнал о большом числе лис,
plus (DoHunt(50)) ;разрешаем отстрел 50 штук
TERMINATE ;произойдет через 25 дней после сигнала

PROCEDURE DoHunt(Pop_Level) Foxes = Foxes - Pop_Level;

PROCEDURE DoRabbit(Zoo_order) Rabbits = Rabbits + Zoo_order;

GENERATE 2000

TERMINATE 1

Популяционная модель

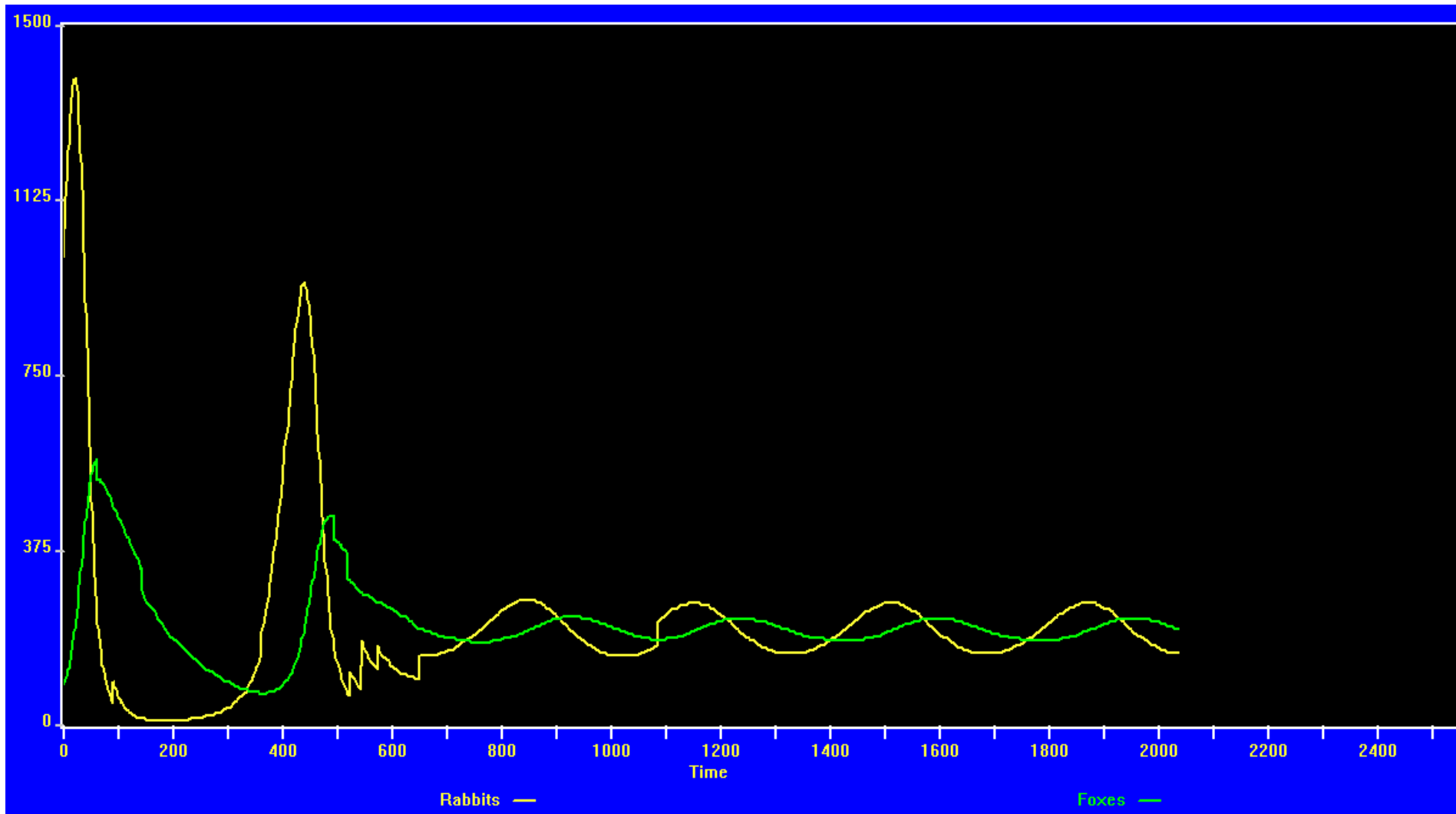


График состояния популяций с учетом влияния дискретных процессов

Популяционная модель

Реальная ситуация:

В 1787 г. впервые завезли кроликов на континент Австралия.

Поедание кроликами травы привело к деградации растительности Австралии, выдуванию и размыванию плодородного слоя почвы.

В результате этого произошло катастрофическое снижение продуктивности пастбищ уже в начале XIX в. Кроме того, у овец случались многочисленные травмы ног из-за кроличьих нор.

Возник риск угрозы овцеводству, в то время одной из главных отраслей экономики Австралии.

Попытки истребления кроликов охотниками не привели к успеху.

В 1950 г. для борьбы с кроликами была применена *вирусная инфекция*, в результате которой за 3 года популяция сократилась в 5 раз.

Модели распространения инфекции

Для аналитической модели распространения заболеваний, после перенесения которых вырабатывается стойкий (*пожизненный*) иммунитет (в т.ч. детские болезни), население разделяют на 3 группы (*compartment*, компартментальная модель):

те, кто восприимчивы к заболеванию (могут заболеть);

те, кто болеет в данный момент;

те, кто переболел и приобрел иммунитет к заболеванию.

Первую группу обозначают буквой S (англ. *Susceptible* – восприимчивый)
вторую – I (англ. *Infectious* – больные, источник инфекции)
третью – R (англ. *Recovered* – выздоровевшие)

Диаграмма переходов между классами популяции



Модель SIR

$$\frac{dS}{dt} = -\beta SI$$

$$\frac{dI}{dt} = \beta SI - \gamma I$$

$$\frac{dR}{dt} = \gamma I$$

$$N = S(t) + I(t) + R(t).$$

Параметр β называется частотой контактов и отражает возможность передачи болезни при контакте между зараженным и восприимчивым. Параметр γ отвечает за переход персоны из класса больных в класс выздоровевших, т.е. γ – частота выздоровлений.

Модель SIR (GPSS)

```
beta_ equ 2      ;заражаемость в популяции  
gama_ equ 0.1    ;вероятность выздоровления(~за 10 дней выздоровеет)
```

```
Rx_ EQU 0        ; число выздоровевших, иммунных людей  
Sx_ EQU 49995    ; число восприимчивых, здоровых людей  
Ix_ EQU 5         ; число больных в начале  
Nx_ EQU 50000    ; популяция людей
```

```
*****
```

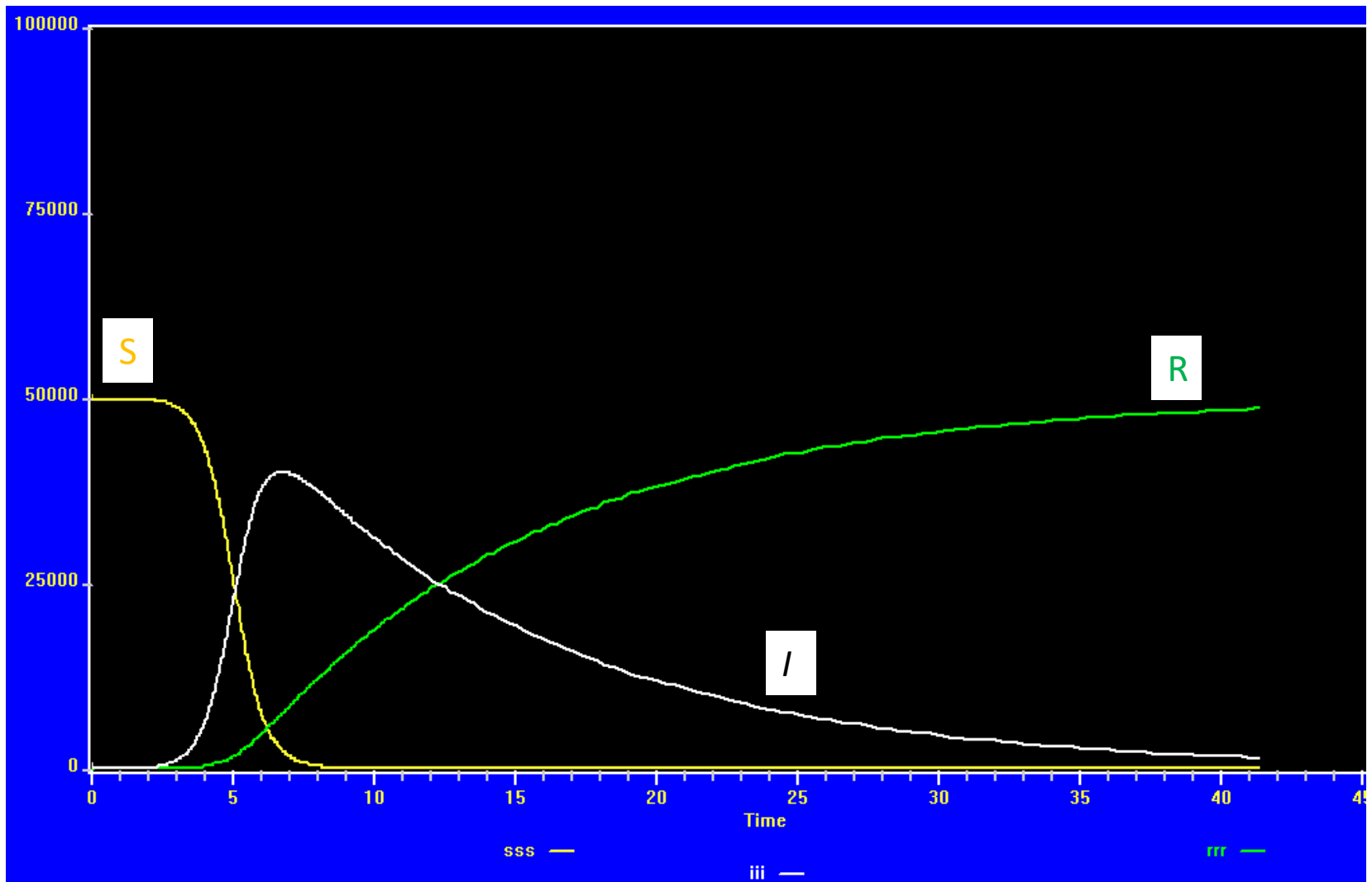
```
Sx_ INTEGRATE (vospriim())  
Ix_ INTEGRATE (Sx_ # Ix_ # beta_ / Nx_ - Ix_ # gama_ )  
Rx_ INTEGRATE (Ix_ # gama_)
```

```
GENERATE 40
```

```
TERMINATE 1
```

```
PROCEDURE vospriim() BEGIN  
    temporary ddd;    if (Sx_ < 0) then Sx_ = 0;  
    if (Sx_ > 1e8) then Sx_ = 1e8;  
    ddd = 0 - Sx_ # Ix_ # beta_/ Nx_ ;    RETURN ddd;  
END;
```

Модель SIR



Уточненная модель SEIR

SEIR - модель с дополнительной «инкубационной» стадией заболевания.

Добавляется класс **E** (Exposed) – т.е. уже заражен, но пока не болен.

SUSCEPTIBLE

$$\frac{dS}{dt} = -\frac{R_e}{T_{inf}} \cdot I \cdot S$$

EXPOSED

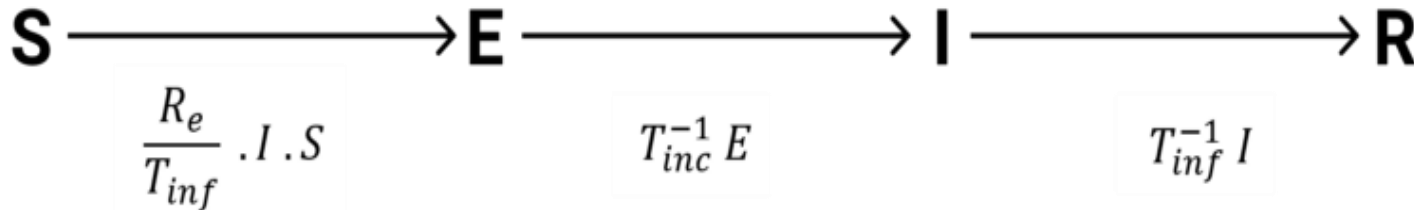
$$\frac{dE}{dt} = \frac{R_e}{T_{inf}} \cdot I \cdot S - T_{inc}^{-1} E$$

INFECTED

$$\frac{dI}{dt} = T_{inc}^{-1} E - T_{inf}^{-1} I$$

REMOVED

$$\frac{dR}{dt} = T_{inf}^{-1} I$$



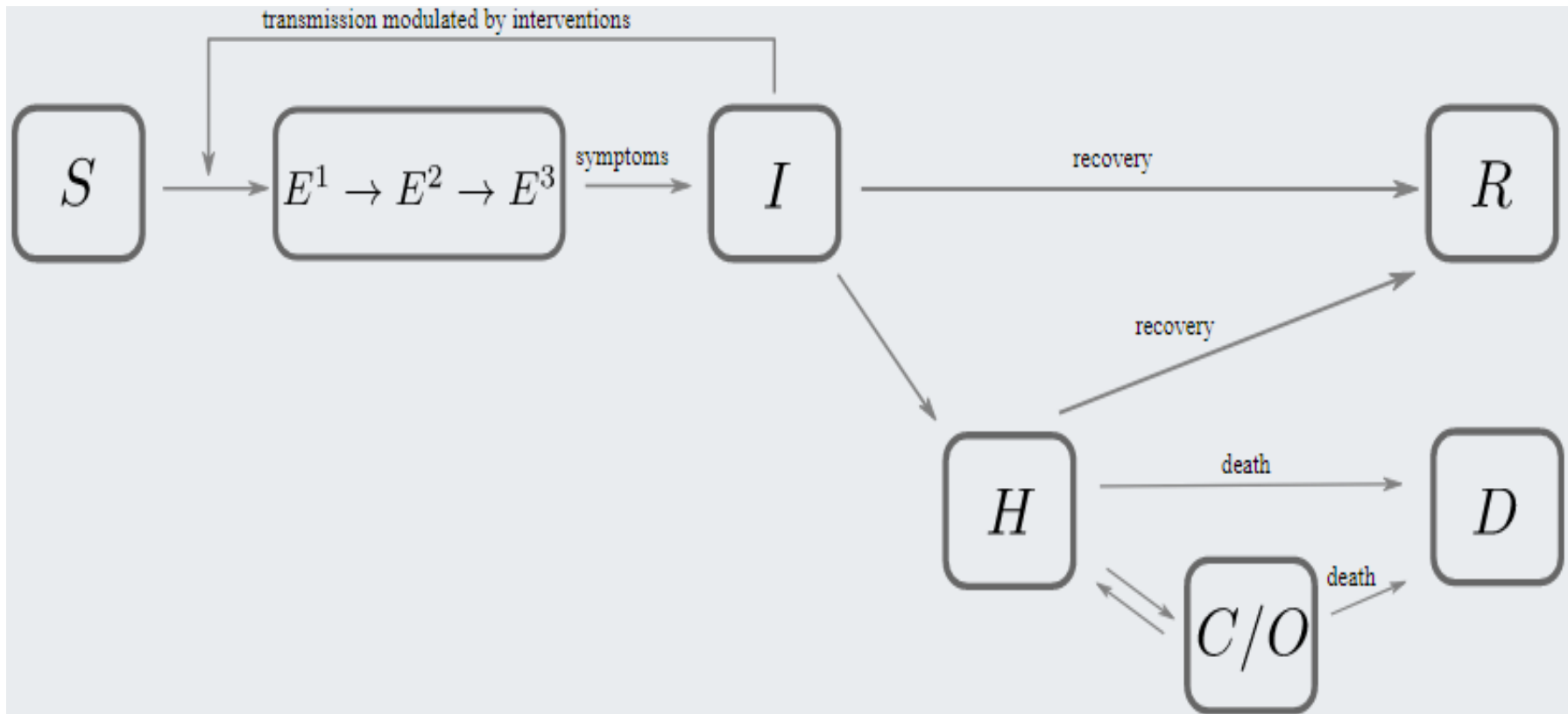
T_{inf} = Infectious period

T_{inc} = Incubation period

R_e = Effective reproduction number

Модель SEnIHmRD

Расширенная **SEIR** модель с дополнительными стадиями инфекционного процесса: добавляются стадии инкубационного периода E^1 - E^2 - E^3 – [n] стадий; H (hospital) – кто болен в больнице; C/O - стадии различных клинических форм заболевания; D (death) – погибшие; R – переболевшие и выздоровевшие.



Модель SE_nIHmRD

$$\frac{dS_a(t)}{dt} = -N^{-1}\beta_a(t)S_a(t) \sum_b I_b(t)$$

$$\frac{dE_a^1(t)}{dt} = N^{-1}\beta_a(t)S_a(t) \sum_b I_b(t) - 3E_a^1(t)/t_l$$

$$\frac{dE_a^2(t)}{dt} = 3E_a^1(t)/t_l - 3E_a^2(t)/t_l$$

$$\frac{dE_a^3(t)}{dt} = 3E_a^2(t)/t_l - 3E_a^3(t)/t_l$$

$$\frac{dI_a(t)}{dt} = 3E_a^3(t)/t_l - I_a(t)/t_i$$

$$\frac{dH_a(t)}{dt} = (1 - m_a)I_a(t)/t_i + (1 - f_a)C_a(t)/t_c - H_a(t)/t_h$$

$$\frac{dC_a(t)}{dt} = c_a H_a(t)/t_h - C_a(t)/t_c$$

$$\frac{dR_a(t)}{dt} = m_a I_a(t)/t_i + (1 - c_a)H_a(t)/t_h$$

$$\frac{dD_a(t)}{dt} = f_a C_a(t)/t_c + p_a H_a(t)/t_h$$

Модели эпид.динамики

- основаны на методе научной аналогии в отображении эпидемического процесса (процесс «переноса» возбудителя инфекции от больных к здоровым) с процессом «переноса» материи (энергии, импульса) в уравнениях математической физики (разработка АМН СССР в 1970-е).

В ходе развития эпидемии среди населения территории, пораженной инфекцией, формируется сложный самоподдерживающийся процесс переноса популяции возбудителя (вируса, микроба) на сообщество восприимчивых людей.

Эпидемиологическое содержание такого процесса связано с отображением, как в календарном времени « t », так и во «внутреннем времени « τ », которое фиксирует развитие инфекционного заболевания у инфицированных людей.

Система уравнений, которая описывает развитие эпидемического процесса, представляет собой систему нелинейных уравнений, схожих с уравнениями гидродинамики.

Модели эпид.динамики

Математическая модель акад.О.В.Барояна и проф.Л.А.Рвачева для эпидемии *гриппа* представляет собой систему нелинейных интегро-дифференциальных уравнений в частных производных с граничными и начальными условиями:

Эпидемический
процесс

$$\begin{aligned} \text{a) } dX(t)/dt &= - [\lambda/P(t)] \times [X(t) \times \int Y(\tau, t) d\tau]; \\ \text{b) } \partial U(\tau, t)/\partial \tau + \partial U(\tau, t)/\partial t &= - \gamma(\tau) \times U(\tau, t); \\ \text{c) } \partial Y(\tau, t)/\partial \tau + \partial Y(\tau, t)/\partial t &= \gamma(\tau) \times U(\tau, t) - \delta(\tau) \times Y(\tau, t); \\ \text{d) } dZ(t)/dt &= \int \delta(\tau) \times Y(\tau, t) d\tau; \end{aligned}$$

Граничные
условия

$$\begin{aligned} \text{a) } U(0, t) &= [\lambda/P(t)] \times [X(t) \times \int Y(\tau, t) d\tau]; \\ \text{b) } Y(0, t) &= 0; \end{aligned}$$

Начальные
условия

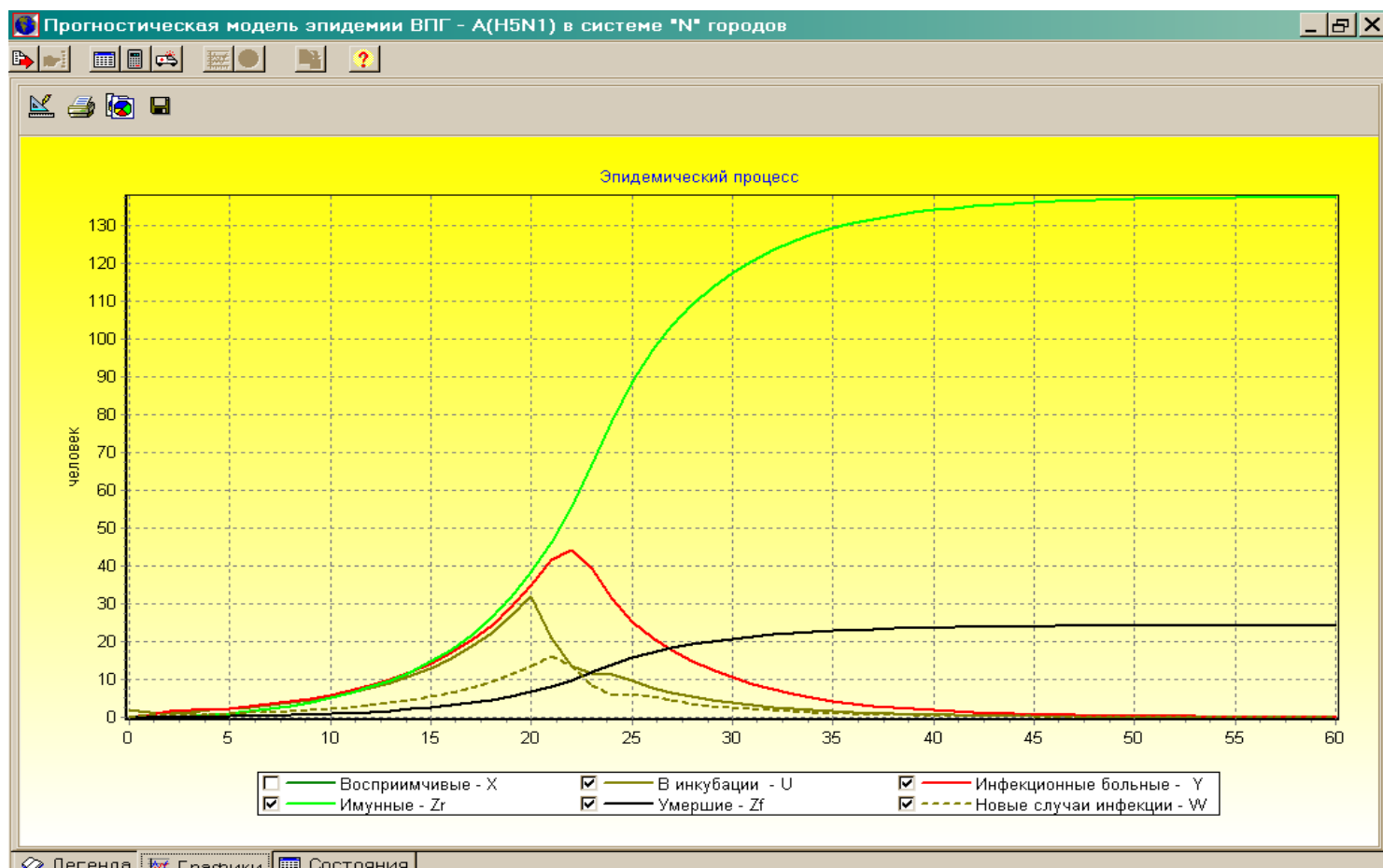
$$\begin{aligned} \text{a) } X(t_0) &= \alpha \times P(t_0); \quad Z(t_0) = (1 - \alpha) \times P(t_0); \\ \text{b) } U(\tau, 0) &= U(\tau); \quad \text{при } 0 < \tau < \tau_u; \\ \text{c) } Y(\tau, 0) &= Y(\tau); \quad \text{при } 0 < \tau < \tau_y, \end{aligned}$$

$t > 0$ – календарное время развития эпидемии (дни); $\tau > 0$ – «внутреннее» время развития инфекционного процесса; λ – средняя частота передачи возбудителя от инфекционных больных Y к восприимчивым X ;

$\gamma(\tau)$ – функция развития периода инкубации; $\delta(\tau)$ – функция развития инфекционного периода; P – население территории, пораженной гриппом;

$\alpha > 0$ – доля восприимчивых среди населения; Z – выздоровевшие.

Модели эпид.динамики



Из модели следует, что за 2 месяца эпидемии в городе (300 тыс ч) погибнет не более 30 человек, а пик «подавленной» эпидемии по числу новых случаев заболевания ожидается на уровне до 15 чел/день на 21-й день после заноса инфекции. Всего в городе заболеет не более 140 чел, а максимальное число больных будет составлять не менее 45 чел, которых можно эффективно изолировать в инфекционных больницах города. Эпидемия полностью купируется за 50 дней с учетом мер противодействия.

Модели эпидемии

Основная проблема таких моделей состоит в том, что предсказания, полученные с их помощью, **очень** чувствительны ко входным параметрам (R_0 , T_{inf} , β , γ , σ ...).

Эти параметры можно измерить лишь *эмпирическим* путем, т.е. собрав достаточную статистику. Статистические данные собираются в тех странах, где вирус уже достаточно сильно распространен.

Применять эти параметры напрямую для моделирования ситуации в другой стране **неверно**, так как при этом не учитываются климат, социальные особенности, средний иммунитет людей, возможные мутации вируса и пр.

Например, высказывается мнение, что прививки от туберкулеза (БЦЖ) могут значительно снизить тяжесть симптомов COVID-19.

Математические модели не учитывают большого количества нюансов, которые потенциально могут повлиять на темпы распространения вируса (в т.ч. наличие мест высокой концентрации людей - школы, магазины, транспорт и т. д.).

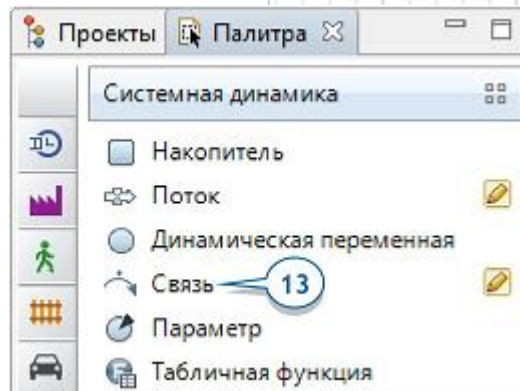
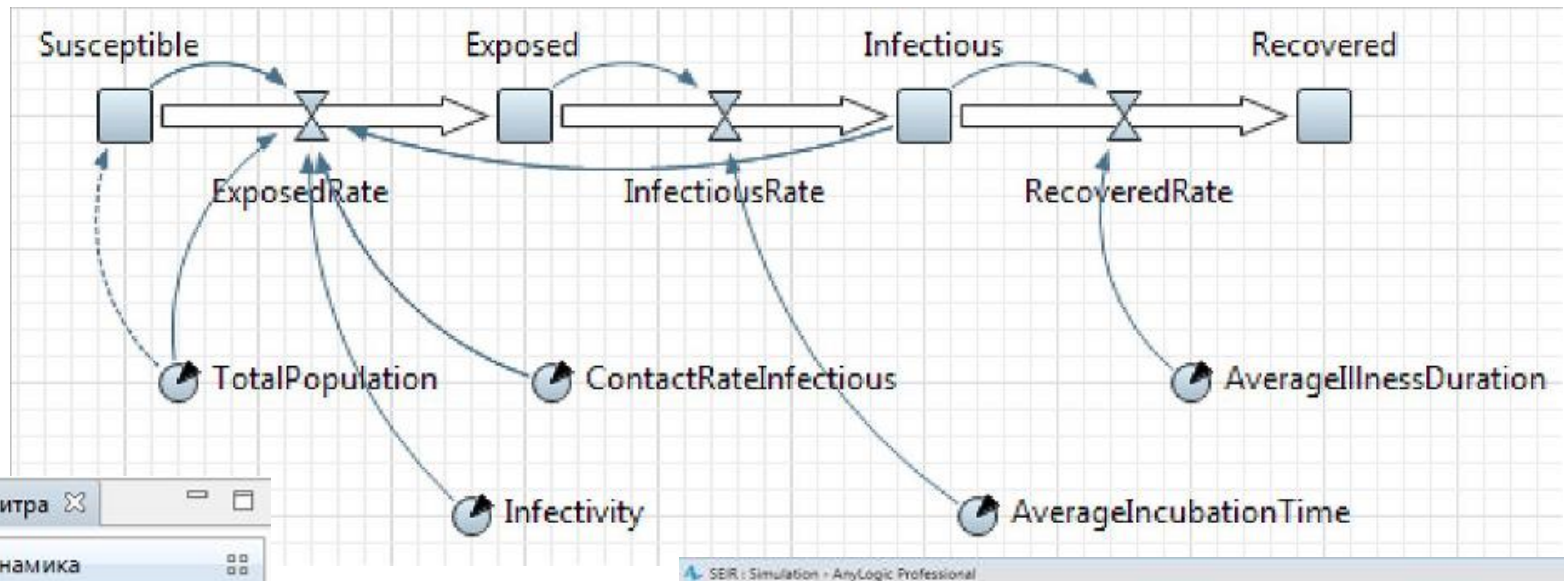
Но на простых моделях понимается общий вид процесса.

Системная динамика

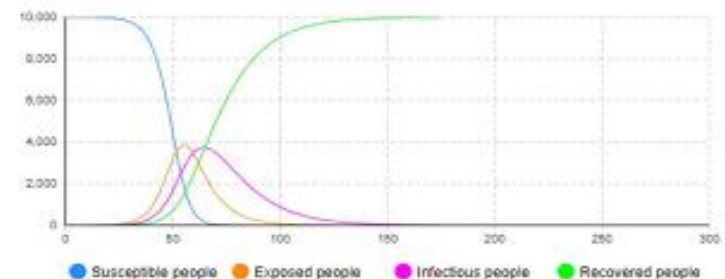
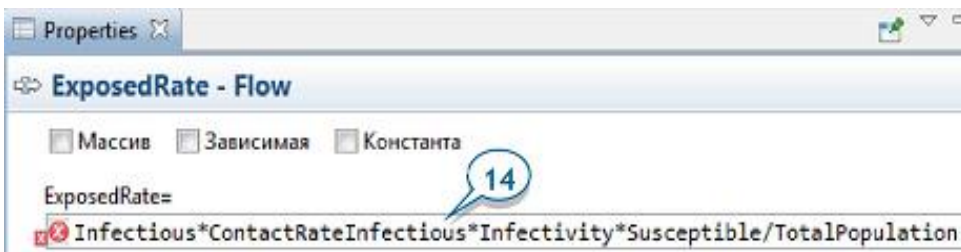
- эффективный и универсальный метод системного анализа окружающего мира с разнообразными возможностями применения в сфере бизнеса, государственного управления, экологии, социологии – там, где требуется принятие управленческих решений в условиях неопределённости, повышенной сложности, масштабности процессов.

Концепция Системной динамики основана на описании взаимодействия потоков и накопителей с применением вентилей.

Модель SEIR (AnyLogic)



SEIR : Simulation - AnyLogic Professional



Агентная модель эпидемии

Пример агентной модели основывается на двумерной среде (пространство X-Y), в которой находятся множество агентов (люди). Каждый агент имеет определенное положение и скорость. Столкновения с границами среды или другими людьми приводят к изменению направления движения агента, они отскакивают от стен и друг от друга («броуновское движение»).

Местоположение агентов в среде выражается вектором положения (x, y) , где x и y - положение, представленное числами с плавающей точкой.

У каждого агента есть скорость (v_x, v_y) .

Все люди подвержены вирусной инфекции и имеют состояния - **здоровые**, **инфицированные**, **иммунные**.

Инфекция снижает коэффициент **здоровья**, причем если этот коэффициент падает ниже 0, то считается, что человек умер от вируса.

Если один из агентов заражен, то при столкновении вирус может передаваться другому агенту.

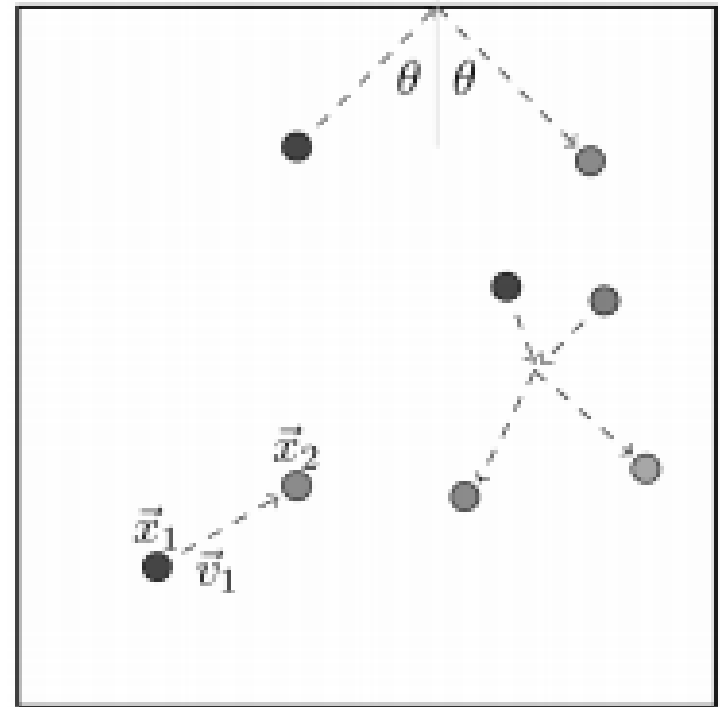
Параметры модели позволяют управлять вероятностью передачи и иммунитетом.

Агентная модель эпидемии

Столкновение агента со стеной приводит к изменению направления движения агента в соответствии с простыми правилами отражения - скорость агента не меняется, но меняется направление. Угол падения ϑ равен углу отражения, т.е. просто изменяется знак одной из составляющих скорости.

При столкновении с другим агентом используется та же концепция, согласно которой агент отскакивает от столкновения, меняя направление, но не скорость.

В модели агенты имеют радиус r . Столкновение происходит, когда центры агентов находятся на расстоянии $2r$. (Но это не кинетическое столкновение, импульс и энергия не сохраняются - при кинетическом столкновении скорость агентов изменяется).



Агентная модель эпидемии

Способ вычисления отражения от стены под углом заключается в повороте системы координат (СК) в горизонтальной плоскости, вычислении отражения и последующем повороте СК обратно в исходное положение. Повернутая СК агента поворачивается на угол α , так чтобы входящий агент приближался к столкновению сверху. Расчет отражения заключается в изменении знака Y-компонента. Затем СК поворачивается на угол α , чтобы вернуться к своей первоначальной ориентации. Каждому агенту на каждом временном шаге (итерации) необходимо изменить свои значения перемещения и взаимодействия с вирусом. Движение определяется уравнением ускоренного движения. Эта функция будет корректировать положение на основе текущего положения, скорости и ускорения. Функция перемещения проверяет пределы окружающей среды и отражает агента от стены, изменив знак одной из скоростей. Если агент заражен, то его уровень здоровья снижается с каждой итерацией.

Агентная модель эпидемии

У каждого агента есть несколько параметров:

- 1) **x**: двумерный массив для позиции, рандомизирован;
- 2) **v**: двумерный массив для скорости, рандомизирован;
- 3) **a**: двумерный массив для ускорения, равно (0,0);
- 4) **alive**: True/False: этот флаг указывает на то, что агент жив;
- 5) **infected**: True/False: этот флаг указывает, что агент заражен;
- 6) **immune**: True/False: этот флаг указывает, что агент иммунен;
- 7) **health**: это параметр запаса здоровья.
- 8) **bottom**: случайное значение [-50...100] для больного, при котором он достигает конца болезни. Если число положительное, то при снижении здоровья агента до этого значения, болезнь заканчивается, и агент становится иммунен. Если число отрицательное, то агент умирает.

У каждого агента есть несколько функций:

- a) **move** - перемещение агента;
- b) **LowerHealth** - снижает уровень здоровья инфицированного человека;
- c) **Cured** - устанавливает параметры больного для лечения болезни;
- d) **Distance** - возвращает расстояние между двумя агентами;
- e) **Collide** - проверяет столкновение двух агентов.

Агентная модель эпидемии

После перемещения агента ему необходимо определить, не столкнулся ли он с другим агентом. Это происходит, когда расстояние между центрами двух агентов меньше, чем R_COMM .

Функция *Distance* вычисляет расстояние от рассматриваемого агента до второго агента, определенного в качестве входного параметра.

Функция *Collide* реализует алгоритм столкновения.

Эта функция вызывается дважды, чтобы изменить поведение обоих участников столкновения. Входными данными для функции *Collide* являются данные другого участника столкновения.

В функции вычисляются расстояния между двумя агентами и угол α . Затем создается матрица поворота, и применяется к СК.

После этого вычисляется отражение, и СК возвращается в исходное состояние.

Также в функции выполняется перенос инфекции.

Агентная модель эпидемии

Функция *Iterate* выполняет одну итерацию проверок агентов.

Итерация включает такие шаги:

1. Переместим агента;
2. Отразим агента от стен, если необходимо;
3. Понизим уровень здоровья зараженных агентов;
4. Определим, удалось ли каким-либо агентам победить вирус;
5. Определим, погибли ли какие-либо агенты;
6. Вычислим последствия столкновения.

В функции *Iterate* параметр people представляет список агентов, перебором которого рассматривается каждый агент.

Не все агенты могут быть инфицированы в ходе первой волны эпидемии, или иммунитет может не сохраняться навсегда. Такое условие потребует к классу Person добавить ещё одну переменную для контроля продолжительности действия иммунитета.

Агентная модель эпидемии

Одна из проблем моделирования заключается в том, что типовой подход ООП к организации кода приводит к замедлению вычислений.

В рассмотренном варианте модели было смоделировано 500 агентов на 250 итераций. Для вычисления потребовалось не менее 200 секунд вычислений. Но с увеличением числа агентов время будет расти в геометрической прогрессии, поскольку каждый агент должен взаимодействовать с другими агентами для определения столкновений.

Это реализуется как двойной вложенный цикл, который значительно увеличивает время вычислений по мере роста числа агентов.

Для увеличения скорости вычислений используем другую структуру данных. Отметим, что каждый агент содержит три типа данных:

- значения с плавающей точкой **float** для положения, скорости, ускорения;
- значения типа **boolean** для флагов "жив", "заражен", "иммунен";
- целочисленные значения **int** для коэф.здоровья и значения bottom.

Агентная модель эпидемии

Создадим три матрицы в формате **numpy array**.

Каждая строка в матрице связана с одним агентом.

Например, 10й агент будет связан со значениями в 10й строке каждой матрицы.

Матрица M содержит значения с плавающей точкой.

Количество строк - это количество агентов, и 6 столбцов для геометрии.

Матрица B содержит тип данных **Boolean** в 3 столбцах.

Матрица I содержит целые значения коэф.здоровья в 2 столбцах.

Эти матрицы создаются в функции **InitVirusMx**.

Функция **IterateM** вычисляет одну итерацию и следует такой же логике.

Запуск моделирования выполняется функцией **RunMatrixVirus**.

Выигрыш во времени работы этой модели составляет 15-20 раз.

Одна из причин в строке цикла вычисления расстояния от агента до всех агентов за одну строку Python: это все еще процесс с вложенным циклом, но один из циклов вложен в функции **numpy**

```
closendx = (dist < 4).nonzero()[0]
```

Entity Component System

При такой компоновке данных аналитику нужно помнить, что представляет собой каждый из столбцов в матрицах.

Строка **n** в каждой матрице соответствует агенту **n**.

Однако нет механизма, который бы на самом деле связывал их воедино. Значения параметров агентов легко могут быть подвержены путанице. Если случайно сдвинуть данные в одной матрице на одну строку, то полностью испортятся все результаты моделирования.

Эта идея организации модели соответствует паттерну **ECS**.

Entity Component System (ECS) - архитектурный паттерн популярный у разработчиков игр и тренажеров.

ECS базируется на 3 определениях: entity (сущность), component (компонент) и system (система).

ECS основывается на принципе *Composition Over Inheritance* (композиция важнее наследования), и является примером *Data Oriented Design* (ориентированного на данные проектирования, **DOD**).

Entity Component System

В ECS «Сущность» - максимально абстрактный объект, условный контейнер для свойств, определяющих чем будет являться эта сущность.

Представляется в виде идентификатора для доступа к данным. Это может быть любой объект в виртуальной среде.

Например, это может быть юнит игрока, кнопка в интерфейсе, событие с данными от одной системы к другой и т.п.

«Сущности» сами по себе не имеют свойств и выступают контейнерами для компонентов.

Поведение объекта в ECS определяется присвоенными объекту свойствами с данными с существующей отдельно логикой обработки.

В некотором приближении, паттерн ECS можно сравнить с базой данных, которая каждый кадр обрабатывает потоком обработчиков, написанных в стиле процедурного программирования.

Entity Component System

В ECS вместо иерархии наследования разделяют данные на изолированные блоки «Компоненты» и набирают из них нужную информацию об игровом объекте, добавляя компоненты на сущности.

«Компоненты» не должны иметь какой-то бизнес-логики в своей обработке и содержат только данные - это важно и является ключевой особенностью по расширению функционала модели.

В ECS данные определяют всё - это главное свойство, которое выделяет его на фоне других подходов к разработке: всё есть данные. И свойства объекта, и его характеристики, и события - всё это данные существующие в ECS-мире.

Entity Component System

В ECS роль обработчиков берут на себя «Системы» - блоки кода, которые каким-либо образом обрабатывают требуемый набор компонентов на сущностях.

«Системы» не содержат никаких локальных данных или ссылок на сущности или компоненты, они служат только для обработки потока отфильтрованных по их условиям сущностей и компонентов, привязанных к сущностям.

«Системы» в ECS - это только логика обработки данных.

Такой подход - разделение логики и данных - позволяет комбинировать любое сложное поведение набором Систем (обработчиков), каждая из которых будет обрабатывать только нужные этой конкретной системе данные на сущностях.

В паттерне ECS можно добавлять и удалять Системы с минимальными изменениями в других Системах, меняя поведение всего игрового мира.

«Система» является конвейерной обработкой всех данных ECS-мира.

Entity Component System

Статья - Всё что нужно знать про ECS

<https://habr.com/ru/articles/665276/>

англоязычный репозиторий

<https://github.com/SanderMertens/ecs-faq>

<https://devlog.hexops.com/2022/lets-build-ecs-part-1/>

YouTube

<https://youtu.be/4sDnBChfV0o>

Telegram

<https://t.me/ecschat>

<https://t.me/unigamethread>